

ParGNN: A Scalable Graph Neural Network Training Framework on multi-GPUs

Junyu Gu^{1,2,*}, Shunde Li^{1,2,*}, Rongqiang Cao^{1,2,*}, Jue Wang^{1,2,†}, Zijian Wang^{1,2}, Zhiqiang Liang^{1,2}, Fang Liu^{1,2}, Shigang Li³, Chunbao Zhou^{1,2}, Yangang Wang^{1,2,†}, Xuebin Chi^{1,2}

¹Computer Network Information Center, Chinese Academy of Sciences, Beijing, China

²University of Chinese Academy of Sciences, Beijing, China

³School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, China
{jygu,lishunde}@cnic.cn, {caorq,wangjue}@sccas.cn, {wangzj,zqliang}@cnic.cn, liufang@sccas.cn, shigangli.cs@gmail.com, {zhoucb,wangyg,chi}@sccas.cn

Abstract—Full-batch Graph Neural Network (GNN) training is indispensable for interdisciplinary applications. Although full-batch training has advantages in convergence accuracy and speed, it still faces challenges such as severe load imbalance and high communication traffic overhead. In order to address these challenges, we propose ParGNN, an efficient full-batch training system for GNNs, which adopts a profiler-guided adaptive load balancing method along with graph over-partition to alleviate load imbalance. Based on the over-partition results, we present a subgraph pipeline algorithm to overlap communication and computation while maintaining the accuracy of GNN training. Extensive experiments demonstrate that ParGNN can not only obtain the highest accuracy but also reach the preset accuracy in the shortest time. In the end-to-end experiments performed on the four datasets, ParGNN outperforms the two state-of-the-art full-batch GNN systems, PipeGCN and DGL, achieving the highest speedup of $2.7\times$ and $21.8\times$ times respectively.

Index Terms—Graph neural network, Full-batch distributed training, Load balancing, Computation and communication overlapping.

I. INTRODUCTION

Graph Neural Networks (GNNs) [1], which are based on graph theory and the graph data structure, have witnessed rapid growth in recent years. GNNs have a wide range of applications in recommender systems [2], chip design [3], energy resource planning [4], etc. They take graphs as input and utilize the structured information of graphs for learning. The core computations in GNN training involve Sparse-Matrix Dense-Matrix Multiplication (SpMM) and Sampled Dense-Dense Matrix Multiplication (SDDMM) operations [5]. The sparse matrix represents the graph topology, while the dense matrices represent the vertex features and model weights. Many real-world datasets can be represented as large-scale graphs, which are difficult to handle on a single computing unit. As the size and complexity of graph data increase, there is growing interest in accelerating GNN training using multiple GPUs [6].

GNN training methods are divided into two categories: mini-batch training and full-batch training. Mini-batch training is a compromise for large-scale graph learning, where subgraphs are sampled and used as mini-batches in each training epoch. Full-batch training, which leverages the entire graph data, generally leads to more accurate and robust models [7].

*These authors contributed equally to work. †Corresponding authors.
ParGNN source code: <https://github.com/echoqujy/ParGNN>

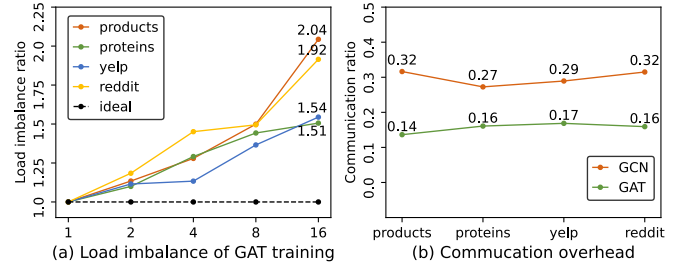


Fig. 1: Load imbalance and communication overhead of distributed GNN training.

However, full-batch training faces challenges such as severe load imbalance and high communication overhead when performed on multiple GPUs [2].

To alleviate the load imbalance of distributed GNN training, existing training systems commonly utilize graph partition tools like METIS [8] to divide the original graph into multiple subgraphs, which are scheduled to run on different GPUs. Those partition tools aim to minimize the number of cut edges while maintaining a balance in the number of vertices or edges between subgraphs. However, the performance of SPMM and SDDMM on GPUs is not only dependent on the number of nonzero elements and the dimension size but also on many other features, such as locality and skew (i.e. the mean, standard deviation, Gini index) [9]. Moreover, specific hardware acceleration implementations based on Tensor Core and others are emerging [10]. Despite these efforts, graph partition methods often fail to achieve true load balancing. Existing studies have not sufficiently focused on quantifying the degree of load balancing achieved. Specifically, they have overlooked an examination of the load imbalance ratio, which is defined as the longest running time divided by the average running time. Fig. 1(a) shows that the load imbalance ratio in METIS-based partition distributed GNN training increases with the number of subgraphs and can exceed 2 in real-world datasets.

The high communication traffic has been recognized as a formidable challenge in the distributed training of GNNs, and several studies have contributed to its reduction. Fig. 1(b) demonstrates that the communication time can occupy 14% to 32% of the training time. DistGNN [11] utilizes a delayed communication strategy to overlap communication with computation on multi-CPU platforms. PipeGCN [12] also hides the communication latency by using the delay strategy and

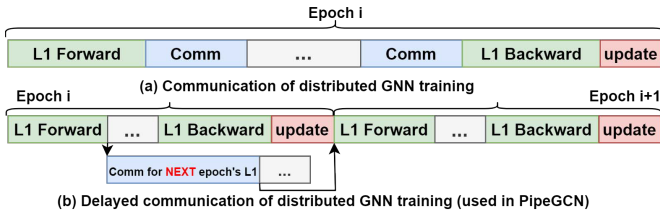


Fig. 2: Communication pattern of distributed GNN training.

breaking the dependency of computation and communication within the graphs on multi-GPU platforms. Although these methods alleviate the communication overhead, they reduce the convergence speed and accuracy of GNN training.

In this paper, we introduce ParGNN, a full-batch training framework for GNNs on multi-GPU platforms, aiming to tackle the challenges of processing large-scale graph data. The key of ParGNN is to adopt an over-partition approach and utilize profiler information in pre-execution to guide workload partition. The over-partition approach not only balances workload but also provides opportunities for true communication and computation overlap while maintaining the accuracy of GNN training. Our main contributions are summarized as follows:

- We propose a Profiler-Guided Adaptive Load Balancing (PGALB) algorithm in conjunction with a two-level graph partition to alleviate load imbalance.
- Based on the over-partition approach, we present a Sub-graph Pipeline (SP) algorithm to overlap communication and computation in each GNN layer in order to accelerate GNN training.
- Extensive experiments of the ParGNN system demonstrate that ParGNN can not only obtain the highest accuracy but also reach the preset accuracy in the shortest time. ParGNN surpasses the two state-of-the-art full-batch GNN systems, PipeGCN and DGL, achieving the highest speedup of $2.7\times$ and $21.8\times$ times respectively.

II. BACKGROUND

Graph Neural Network. Graph neural network is stacked by multiple layers of graph neural network layers. In each layer of graph neural network, the features on each vertex are updated by aggregating the features of their neighboring vertices along topological weights. Given the output feature matrix H^{l-1} on layer $l-1$, the forward propagation process of layer l is Equation (1), where W^l is the weight matrix, σ is the activation function, and A is a sparse topological weight matrix [13].

$$H^l = \sigma(AH^{l-1}W^l) \quad (1)$$

The most expensive operations are SpMM in $A \times H^{l-1}W^l$. For GNN models that update the topological weights at each layer (such as GAT [14]), they carry out the updates via SDDMM-like operations in Equation (2).

$$A' = A \odot ((H^{l-1}W_{src}^l) \times (H^{l-1}W_{dst}^l)^T) \quad (2)$$

where W_{src}^l and W_{dst}^l represent the network weights applied to the features of the source and destination vertices on the edges, respectively.

Distributed Full-batch GNN Training. To facilitate the training of full-batch GNNs on large-scale graphs, graph partition techniques are commonly employed. Specifically, the original graph is partitioned into multiple subgraphs, and each subgraph is allocated in GPU memory. This approach effectively alleviates memory constraints. By simultaneously training each subgraph, the overall training is significantly accelerated.

Communication Pattern. Since each GPU's subgraph contains only a portion of the entire graph's data, it is necessary to communicate and update the vertex feature information of the subgraph between two layers of GNN training. As shown in Fig. 2(a), during distributed GNN training, when finishing the calculation of one layer of GNN and starting the calculation of the next layer, the currently updated information is promptly communicated to other GPUs. In Fig. 2(b), the communication is delayed by one epoch, that is, the L1 update of epoch i is communicated and used for L1 of epoch $i+1$.

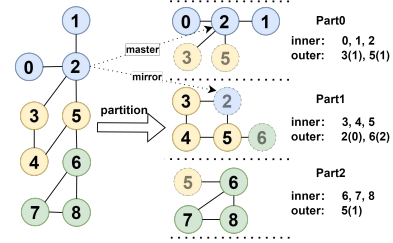


Fig. 3: Inner and outer vertices.

Fig. 3 shows an example where an original graph is divided into three subgraphs and vertices in each subgraph are divided into inner and outer vertices. The inner vertices, such as vertices ③, ④, and ⑤, are assigned to Part1 by the partitioner. The outer vertices, such as vertices ② and ⑥ in Part1, connect to the inner vertices but are assigned to other subgraphs. The outer vertices are copies of the inner vertices of other subgraphs. These copied inner vertices are also called the master vertices of the outer vertices in other subgraphs, while the outer vertices are named the mirrors of the master vertices. The inner vertices in each subgraph are updated through local calculations. However, the outer vertices (mirrors) are updated by receiving copies of the corresponding inner vertices (masters) from other subgraphs. For example, in subgraph Part1 in Fig. 3, the updates for inner vertices ③, ④, and ⑤ are obtained through aggregation calculations involving all vertices in the subgraph (vertices ②, ③, ④, and ⑤, and ⑥). Meanwhile, outer vertex ② receives its update from Part0, and outer vertex ⑥ receives its update from Part2.

III. SYSTEM DESIGN

ParGNN is a framework specifically designed for training large-scale graph neural networks on multiple GPUs. It leverages the powerful performance of GPU clusters to handle the growing amount of graph neural network data and is committed to addressing future scalability challenges. We name it Parallel Graph Neural Network (ParGNN). ParGNN uses full-batch processing to ensure the integrity of training information. At the same time, the entire graph is partitioned

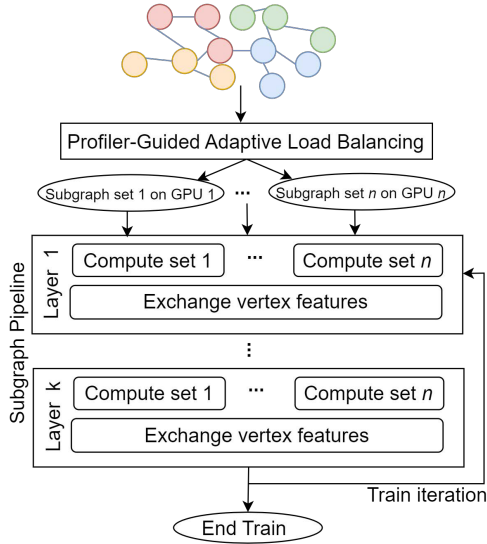


Fig. 4: Overview of ParGNN system.

and mapped into GPU memory. Different from other GNN training frameworks, ParGNN pays attention to the actual load of graph data on different GPUs and reconsiders the performance evaluation of sparse matrix operators required for GNN training on GPUs, aiming to enhance load balancing effects and improve training efficiency.

ParGNN is mainly composed of two parts, PGALB and SP. The purpose of PGALB is to allocate load-balanced data partitions for GPUs, while SP provides computation and communication overlap when training partitioned graph data after allocation.

Fig. 4 shows the workflow of ParGNN performing distributed full-batch GNN training on n GPUs. The original graph data is divided into n subgraph sets by PGALB's two-stage graph partitioning strategy. Each subgraph set contains several subgraphs and is mapped to one GPU. In the training stage, each GPU is responsible for training one subgraph set. In each GNN layer, ParGNN enables SP. This implements the pipeline overlapping of vertices features update computation and communication between subgraph sets of different GPUs. After completing the forward and backward computation of k layers of GNN in one epoch, ParGNN will check whether to continue iterative calculation or end training according to preset training end conditions, such as reaching the specified number of epochs or loss value.

IV. PROFILER-GUIDED ADAPTIVE LOAD BALANCING

The sparsity of graph data causes a significant load imbalance during the execution process of distributed GNN. To alleviate this severe load imbalance, it is essential to partition the original graph data effectively, ensuring that computation tasks are distributed as evenly as possible among different computing units. However, as mentioned in Section 1, the method of evaluating computational load based on the number of vertices or edges is crude. The computational performance of sparse matrices largely depends on the sparsity of vertices and the density of memory access. Therefore, we propose a

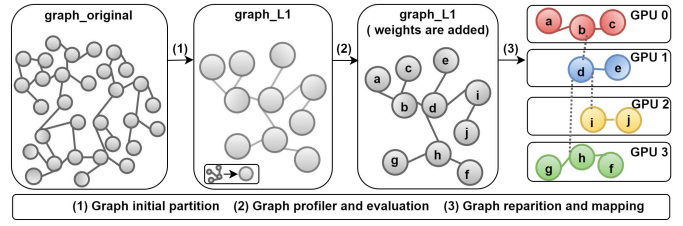


Fig. 5: Workflow of PGALB.

novel profiler-guided two-level partition approach guided to achieve adaptive load balancing in distributed GNN training.

The workflow of two-level adaptive load balancing when used for the data partition is described in Fig. 5 with the following three steps:

(1) Graph initial partition. The raw graph data is processed to initialize the data graph (`graph_original`) for subsequent partition and evaluation. Then, the `graph_original` is coarsened into a small-sized graph (`graph_L1`) to facilitate the evaluation of computational workload and load balancing. In `graph_L1`, each vertex represents a subgraph composed of multiple vertices from `graph_original`. The partition is based on the principle of having an approximately equal number of vertices and minimizing the number of cut edges to reduce inter-subgraph communication. This enables us to effectively control the scale of the graph, facilitating subsequent adjustments to load balancing.

(2) Graph profiler and evaluation. During the graph evaluation process, we evaluate all subgraphs of `graph_L1` in a pre-executed manner to assess the computational load of the entire graph. We construct a GNN model that contains various dense and sparse operations that need to be performed during real training. By sequentially executing the model's forward calculation process for each subgraph and accurately recording the actual time for each execution, we can measure the computational load of each subgraph. Then, we assign weights to each vertex and edge in `graph_L1` based on the calculated time and the number of cut edges.

(3) Graph repartition and mapping. We partition the subgraphs to each GPU based on the subgraph weights on `graph_L1`. The number of partitions is equal to the number of GPUs. The goal of partitioning `graph_L1` is to achieve as much load balancing of computational tasks as possible across the partitions. In addition, the weights of edges between vertices are also considered during the mapping process. By minimizing the number of cutting edges, data communication between partitions can be effectively reduced. Each GPU is assigned multiple subgraphs (vertices in `graph_L1`). Thus, we have completed the data preparation for each GPU and can proceed with the model training process. During the training phase, for each GPU, we sequentially train these subgraphs.

By employing the above workflow, we can effectively reduce the load imbalance during distributed GNN training. We can achieve the optimal utilization of computational resources to enhance system efficiency and performance. The frameworks using one time graph partition have certain drawbacks. In case of a slight change in the original graph or a change in the number of underlying hardware processing units, they

must carry out a new partition of the original graph, and this process is very time consuming. Our strategy is to carry out large graph partitioning with time-consuming only once in the initial partition. The repartition and mapping step acts on the small graph_L1, and its actual consumption can be neglected. Meanwhile, for the computational load evaluation of each subgraph (vertex in graph_L1), it can be reassigned according to the actual situation during training and adapted to find a more balanced subgraph-GPU mapping relationship. In addition, it can be quickly adjusted according to the changes in the computing resources needed. Specifically, our algorithm is more suitable for continuous learning of GNNs.

V. SUBGRAPH PIPELINE ALGORITHM

ParGNN adjusts the workload between GPUs by allocating multiple subgraphs on one GPU, providing the opportunity to overlap computation and communication. However, two challenges need to be solved: (1) the number of subgraphs between different GPUs is inconsistent; (2) there is temporal dependence between subgraphs.

We assume GPU i has k_i subgraphs. And G_{ij} represents the j -th subgraph on GPU i , which adopts a compressed storage format. Each subgraph triggers a communication operation after its local computation is completed. The number of subgraphs on each GPU is different, so the number of communications on different GPUs is also different. The temporal dependence between subgraphs is manifested as follows: after a subgraph G_{ij} completes its local computation and updates the inner vertices, it needs to send the features of master vertices to the mirrors of other subgraphs. At the same time, the outer vertices need to wait for the reception from remote master vertices which have completed the update and sent out. However, the subgraphs on each GPU are computed sequentially. The master vertices of subgraph G_{ij} need to be sent to the subgraph $G_{i'j'}$ ($i' \neq i, j' \neq j$) on other GPUs, but the $G_{i'j'}$ may not be in the process of execution. It might have been executed ($j' < j$) or still be waiting to be executed ($j' > j$).

SP algorithm designs a structure that separates the communication sending and update retrieving operations of subgraph features to uniformly solve the above two challenges. For subgraph G_{ij} , its corresponding set of outer vertices is V_{ij}^o . The set of outer vertices that the GPU i wants to retrieve is $V_i^o = V_{i0}^o \cup \dots \cup V_{ik_i-1}^o$. SP constructs a feature receiving buffer B_i based on the outer vertex set of all subgraphs V_i^o on the current GPU i for receiving all vertex features from other GPUs. Although all communications use the same receiving buffer B_i , each communication fills a different part of the buffer. The all2allv primitive is used to implement irregular peer-to-peer communication between GPUs. After all communications are completed, each subgraph G_{ij} retrieves its outer vertices features from buffer B_i for update. k_{max} represents the maximum number of subgraphs across all GPUs, indicating the number of communications required to fill all the feature receiving buffers on all GPUs. It should be noted that due to some GPUs having less than k_{max} subgraphs, there

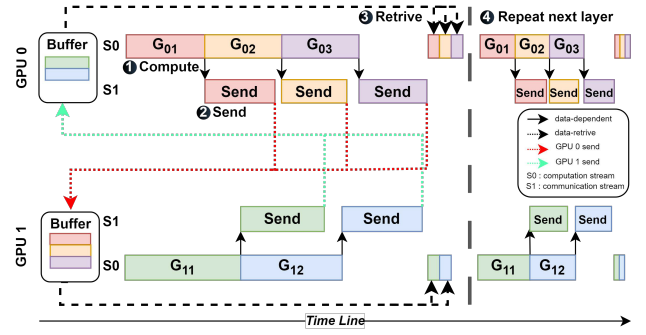


Fig. 6: Example for SP workflow. For subgraph G_{01} on GPU0, it first conducts local computations ①. After the computations are completed, the system immediately issues the asynchronous sending of the features of its master vertex ②. The sending process overlaps with the computation process of subsequent subgraphs. After the receiving buffer is full, G_{01} updates the outer vertices by retrieving the remote feature from the buffer ③. Then, G_{01} finishes all vertex updates and gets ready for the next computation ④. are additional $k_{max} - k_i$ communication calls with keeping receiving but sending empty content.

Fig. 6 shows an example of SP execution when training on two GPUs. There are three subgraphs on GPU 0 and two subgraphs on GPU 1. SP utilizes the multi-stream execution feature on the GPU to configure two CUDA streams for each GPU during computation. There are the computation stream and the communication stream for computation and communication tasks respectively. The computation and asynchronous communication calls of the three subgraphs on GPU 0 are sequentially interleaved. Under the control of the CUDA stream, the communication time of G_{01} and G_{02} can be overlapped by the computation time of G_{02} and G_{03} . The data on GPU 0 is communicated to the receiving buffer on GPU 1. Similarly, the data that GPU 0 wants is also communicated to the local buffer. After the buffer is full (that is, all the desired feature data has been received), the three subgraphs sequentially retrieve the updated features of the outer vertices from the buffer and start the computation of the next network layer.

The intermediate variables F'_{ij} and F''_{ij} are introduced to represent the computation and update process of the vertex features F_{ij} for subgraph G_{ij} . The forward propagation execution process of SP is shown in Equation (3) and (4).

$$F'_{ij} = \text{ComputeGraph}(G_{ij}, F_{ij}) \quad (3)$$

$$F''_{ij} = \text{UpdateGraph}(B_i, F'_{ij}) \quad (4)$$

Since Equation (4) is an indexing filling process, i.e., $\frac{\partial(F''_{ij})}{\partial(F'_{ij})} = 1$. According to the chain rule for differentiation [15], the backward propagation for calculating the gradient of the loss with respect to F_{ij} is as shown in Equation (5).

$$\begin{aligned} \frac{\partial(\text{loss})}{\partial(F_{ij})} &= \frac{\partial(\text{loss})}{\partial(F''_{ij})} \cdot \frac{\partial(F''_{ij})}{\partial(F'_{ij})} \cdot \frac{\partial(F'_{ij})}{\partial(F_{ij})} \\ &= \frac{\partial(\text{loss})}{\partial(F'_{ij})} \cdot 1 \cdot \frac{\partial(F'_{ij})}{\partial(F_{ij})} \\ &= \frac{\partial(\text{loss})}{\partial(F'_{ij})} \cdot \frac{\partial(\text{ComputeGraph}(G_{ij}, F_{ij}))}{\partial(F_{ij})} \end{aligned} \quad (5)$$

The SP algorithm provides a feature computation and communication overlap pipeline for distributed GNN training. Note that although the SP algorithm is derived based on the characteristic of ParGNN storing multiple subgraphs on one GPU, the architecture that separates communication sending and feature retrieval can be easily extended to the common case where there is only a single subgraph on a GPU. In this case, only row splitting of the topological adjacency matrix corresponding to the subgraph is needed, and then the asynchronous communication overlap ability provided by the SP algorithm can be used.

VI. EVALUATION

Experiment environment. We conduct experiments on DGX-H100 server. Which is equipped with an Intel Xeon Platinum 8358P 48-core processor, eight NVIDIA H100 80 GB GPUs with NVLINK connect, and 2048 GB host memory. We carry out the experiments using PyTorch 2.3.1 [16] and DGL 2.4.0 [5] with cuda-12.1.

TABLE I: Dataset information.

Dataset	products	proteins	yelp	reddit
Vertices	2,449,029	132,534	716,847	232,965
Edges	61,859,140	39,561,252	13,954,819	114,615,892
Features dim	100	128	300	602
Categories	47	112	100	41

Datasets. We use 4 public and authoritative real-world graph datasets. The details of these datasets are listed in Table I. Ogbn-Products (products) is an Amazon products co-purchasing network [17]. Ogbn-proteins (proteins) is a protein-protein association networks [18]. Yelp is a network dataset that classifies business types based on user reviews and social relationships [19]. Reddit is a community network of online posts based on user comments [20].

Models and Parameters. We use GCN [13] and GAT [14] as the experimental backbone networks for classification tasks in 300 epochs. Each model is stacked with 3 layers and the hidden dimension is 256. For the GAT model, the number of attention heads is 4. We train all datasets and models on 4 GPUs in 300 epochs with the following experimental parameters: Dropout [21] rate is 0.5, the Adam [22] optimizer with a learning rate of 0.001, weight decay is 0.0001, and the random seed is set to 2024.

A. Load Balancing Evaluation

Table II presents the load balancing performance comparisons between METIS and the PGALB method based on METIS' initial partitions. METIS uses the edge-cut partitioning algorithm to partition four datasets, and the number of subgraphs partitioned by PGALB is four times that of the number of GPUs. Compared to the execution time of the METIS partition, the increase in the execution time of load balancing adjustment using PGALB is within 1.5%, indicating that the increase in time for PGALB is negligible. At the same time, the PGALB algorithm reduces the load imbalance ratio from 6.59% to 43.38%. Since the graph processed in the graph profiler and repartition stages is relatively small

TABLE II: PGALB v.s. METIS. The upper part shows the partitioning time, while the lower part indicates the degree of load imbalance.

Dataset	products	proteins	yelp	reddit
METIS time(s)	36.07	17.73	4.81	26.22
PGALB time(s)	36.26	17.88	4.87	26.51
increase ratio(%)	0.52%	0.85%	1.33%	1.12%
Load imbalance evaluation				
METIS (%)	118.27%	126.75%	112.45%	145.98%
PGALB (%)	105.93%	102.31%	105.85%	102.59%
reduction ratio(%)	12.34%	24.45%	6.59%	43.39%

in scale, PGALB has less additional execution time after the initial partition, enabling PGALB to perform rapid adaptive load balancing adjustments.

B. Distributed GNN System Evaluation

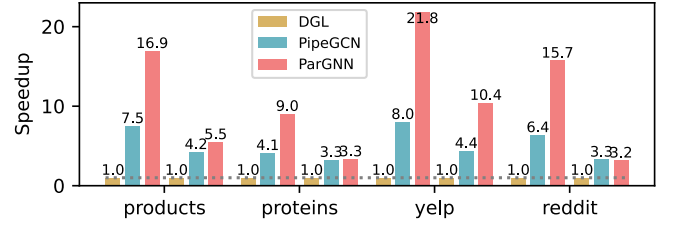


Fig. 7: Execution time speedup of three GNN systems for all datasets. In each dataset, the left three columns represent the speedup on the GCN model, and the right three columns are also for the GAT model.

We select two state-of-the-art distributed GNN training systems for comparison, namely DGL [5] (the most widely used industrial GNN system) and PipeGCN [12] (a system that achieves speedup through the delaying strategy of the communication pipeline).

We present the speedup of the average epoch training time of ParGNN and PipeGCN relative to DGL in Fig. 7. In general, the execution efficiency of ParGNN is better than both PipeGCN and DGL, achieving the highest speedup of $2.7\times$ and $21.8\times$ times respectively, while DGL has the poorest computing efficiency on all datasets. The acceleration effect of ParGNN on DGL is $9.0\times$ to $21.8\times$ on GCN and $3.2\times$ to $10.4\times$ on GAT. For PipeGCN, ParGNN can generally obtain about a $2\times$ acceleration effect on GCN and similar speedup in GAT.

We present the training accuracy on products under three different systems in Fig. 8, which shows the variation of test accuracy with the increase in training epochs. The three systems are converging to a very similar accuracy on the two models. The training accuracy of the last 100 epochs is enlarged in the figure, and the highest accuracy reached

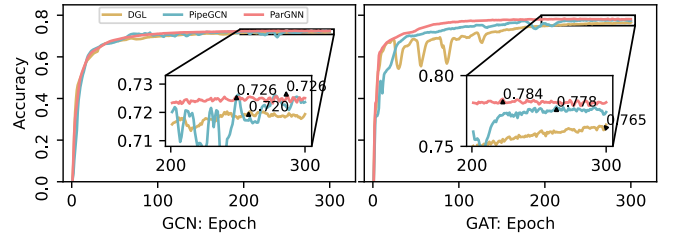


Fig. 8: Accuracy variation with epochs for the products dataset. The left subfigure shows the results of GCN, and the right one presents those of GAT.

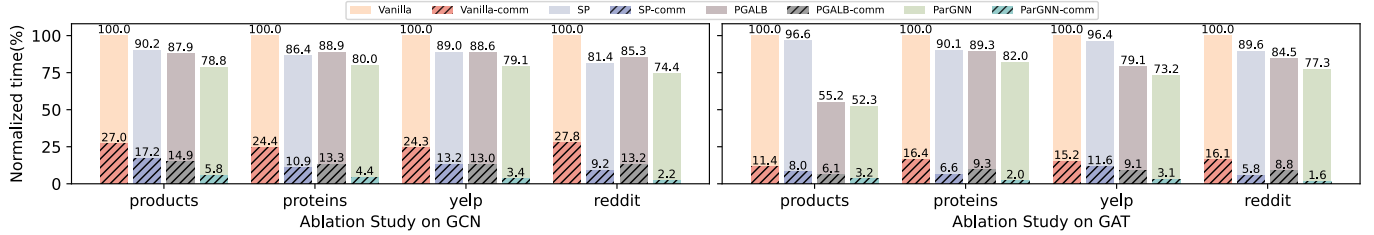


Fig. 9: Ablation study of ParCGNN for all datasets. The left and right subfigures respectively show the normalized time of GCN and GAT based on Vanilla average epoch time for each dataset, including average epoch and communication time.

by different systems is marked respectively. It shows that the difference between them is in a bound of 2.5%. This indicates that ParGNN can have a stable convergence effect like PiPeGCN and DGL.

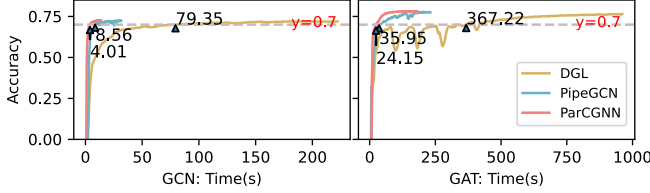


Fig. 10: Time-To-Accuracy for the products dataset. The left subfigure shows GCN's performance, and the right one presents GAT's.

We also provide the Time-To-Accuracy indicator. It can intuitively reflect the efficiency of the model in achieving the expected accuracy and offer a more comprehensive perspective for evaluating the system performance [23]. As shown in Fig. 10, we set the predetermined accuracy as 0.70 on the products dataset. In GCN, ParGNN reaches the target accuracy at 4.01 seconds. PipeGCN and DGL take 8.56 seconds and 79.35 seconds, respectively. In GAT, ParGNN is also the fastest, taking 24.15 seconds. DGL takes 35.95 seconds. Instead, PipeGCN takes the longest time of 367.22 seconds. ParGNN can converge to the predetermined accuracy the fastest on both models.

Overall, ParGNN can not only obtain the highest accuracy but also reach the preset accuracy in the shortest time. This indicates that ParGNN presents a more stable training performance compared to DGL and PipeGCN.

C. Ablation Evaluation of ParGNN

ParGNN is divided into the following four versions for the ablation experiment analysis to analyze the contributions of the two factors, namely PGALB and SP:

- (a) ParGNN: This is the full-functional GNN training system that is optimized with both PGALB and SP.
- (b) PGALB: This is the version of ParGNN where only the PGALB optimization is enabled.
- (c) SP: This is the version of ParGNN where only the SP optimization is enabled.
- (d) Vanilla: This is the version of the multi-subgraph distributed GNN training system after discarding both optimization methods.

We normalize the epoch execution time and communication time of each dataset on the specific model based on the execution results of Vanilla to better show the effects of different datasets on different models. The purpose is to fully present the performance improvement of different optimization

methods in communication and overall efficiency, as shown in Fig. 9.

The performance improvement brought by PGALB is more concentrated on the GAT model. As stated in Equation (2), the computational load of GAT is greater than that of GCN. When the computational load ratio is heavier, the improvement in computational efficiency brought by load balancing is higher. In Fig. 1, it is shown that the computational time proportion during GAT training is on average 14.25% higher than that of GCN. In the ablation experiments, the average improvement (Vanilla $\rightarrow$ PGALB) in effect brought by PGALB's adaptive load balancing for GAT is 22.95%, while the average improvement in effect obtained on GCN is 12.31%.

The computation and communication overlap provided by SP is also very obvious. SP can reduce the time proportion in GCN from an average of 12.62% to 3.95%, and reduce the time proportion in GAT from an average of 7.99% to 2.49% (PGALB $\rightarrow$ ParGNN). The reduction ratios of time are 68.70% and 68.86%, respectively. This gain is in line with the preset situation where each GPU is responsible for an average of four subgraphs. If the communication of three subgraphs is completely overlapped and the communication time for each subgraph is the same, the communication time will be reduced by approximately 75% in an ideal state. And the actual gain is already close to 70%, which shows the communication overlap effect of SP is good.

In general, ParGNN has a 20.0% to 25.6% improvement in performance on GCN and an 18.0% to 47.7% improvement in performance on GAT. Both PGALB and SP can effectively improve the performance of distributed GNN training.

VII. CONCLUSION

In this paper, we introduce ParGNN, a full-batch GNN training system for multi-GPU platforms. ParGNN adopts the PGALB algorithm with a two-level graph partition to reduce workload imbalance. ParGNN enables multiple subgraphs to be trained on the same GPU and employs the SP algorithm for overlapping computation and communication while maintaining accuracy, outperforming state-of-the-art frameworks such as DGL and PipeGCN.

ACKNOWLEDGMENT

This work is supported by the National Key Research & Development Program of China (Grant No. 2023YFB4502303), and the Strategic Priority Research Program of Chinese Academy of Sciences (Grant No. XDB0500103).

REFERENCES

- [1] F. Scarselli, M. Gori, A. C. Tsoi, M. Hagenbuchner, and G. Monfardini, "The graph neural network model," *IEEE Transactions on Neural Networks*, vol. 20, no. 1, pp. 61–80, 2009.
- [2] H. Lin, M. Yan, X. Ye, D. Fan, S. Pan, W. Chen, and Y. Xie, "A comprehensive survey on distributed training of graph neural networks," *Proceedings of the IEEE*, 2023.
- [3] A. Mirhoseini, A. Goldie, M. Yazgan, J. W. Jiang, E. M. Songhori, S. Wang, Y.-J. Lee, E. Johnson, O. Pathak, A. Nazi, J. Pak, A. Tong, K. Srinivasa, W. Hang, E. Tuncer, Q. V. Le, J. Laudon, R. Ho, R. Carpenter, and J. Dean, "A graph placement methodology for fast chip design," *Nature*, vol. 594, pp. 207–212, 2021. [Online]. Available: <https://api.semanticscholar.org/CorpusID:235395490>
- [4] T. Yao, J. Wang, Y. Wang, P. Zhang, H. Cao, X. Chi, and M. Shi, "Very short-term forecasting of distributed pv power using gstann," *CSEE Journal of Power and Energy Systems*, 2022.
- [5] M. Wang, D. Zheng, Z. Ye, Q. Gan, M. Li, X. Song, J. Zhou, C. Ma, L. Yu, Y. Gai, T. Xiao, T. He, G. Karypis, J. Li, and Z. Zhang, "Deep graph library: A graph-centric, highly-performant package for graph neural networks," 2020.
- [6] D. Zheng, X. Song, C. Yang, D. LaSalle, and G. Karypis, "Distributed hybrid cpu and gpu training for graph neural networks on billion-scale graphs," 2022.
- [7] Z. Jia, S. Lin, M. Gao, M. Zaharia, and A. Aiken, "Improving the accuracy, scalability, and performance of graph neural networks with roc," in *Proceedings of Machine Learning and Systems*, I. Dhillon, D. Papailiopoulos, and V. Sze, Eds., pp. 187–198.
- [8] G. Karypis and V. Kumar, "A fast and high quality multilevel scheme for partitioning irregular graphs," *SIAM J. Sci. Comput.*, vol. 20, no. 1, pp. 359–392, dec 1998.
- [9] S. Yesil, A. Heidarshenas, A. Morrison, and J. Torrellas, "Wise: Predicting the performance of sparse matrix vector multiplication with machine learning," in *Proceedings of the 28th ACM SIGPLAN Annual Symposium on Principles and Practice of Parallel Programming*, ser. PPoPP '23. New York, NY, USA: Association for Computing Machinery, 2023, pp. 329–341. [Online]. Available: <https://doi.org/10.1145/3572848.3577506>
- [10] R. Fan, W. Wang, and X. Chu, "Dtc-spm: Bridging the gap in accelerating general sparse matrix multiplication with tensor cores," in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, 2024, pp. 253–267.
- [11] V. Md, S. Misra, G. Ma, R. Mohanty, E. Georganas, A. Heinecke, D. Kalamkar, N. K. Ahmed, and S. Avancha, "Distgcn: Scalable distributed training for large-scale graph neural networks," in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC '21. New York, NY, USA: Association for Computing Machinery, 2021. [Online]. Available: <https://doi.org/10.1145/3458817.3480856>
- [12] C. Wan, Y. Li, C. R. Wolfe, A. Kyrillidis, N. S. Kim, and Y. Lin, "Pipegcn: Efficient full-graph training of graph convolutional networks with pipelined feature communication," 2022.
- [13] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *arXiv preprint arXiv:1609.02907*, 2016.
- [14] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," *International Conference on Learning Representations*, 2018.
- [15] N. Ketkar, J. Moolayil, N. Ketkar, and J. Moolayil, "Automatic differentiation in deep learning," *Deep Learning with Python: Learn Best Practices of Deep Learning Models with PyTorch*, pp. 133–145, 2021.
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Red Hook, NY, USA: Curran Associates Inc., 2019.
- [17] W.-L. Chiang, X. Liu, S. Si, Y. Li, S. Bengio, and C.-J. Hsieh, "Cluster-gcn: An efficient algorithm for training deep and large graph convolutional networks," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, pp. 257–266. [Online]. Available: <https://doi.org/10.1145/3292500.3330925>
- [18] D. Szklarczyk, A. L. Gable, D. Lyon, A. Junge, S. Wyder, J. Huerta-Cepas, M. Simonovic, N. T. Doncheva, J. H. Morris, P. Bork *et al.*, "String v11: protein–protein association networks with increased coverage, supporting functional discovery in genome-wide experimental datasets," *Nucleic acids research*, vol. 47, no. D1, pp. D607–D613, 2019.
- [19] H. Zeng, H. Zhou, A. Srivastava, R. Kannan, and V. Prasanna, "Graph-saint: Graph sampling based inductive learning method," *arXiv preprint arXiv:1907.04931*, 2019.
- [20] W. Hamilton, Z. Ying, and J. Leskovec, "Inductive representation learning on large graphs," *Advances in neural information processing systems*, vol. 30, 2017.
- [21] N. Srivastava, G. E. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *J. Mach. Learn. Res.*, vol. 15, pp. 1929–1958, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6844431>
- [22] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.
- [23] C. Coleman, D. Kang, D. Narayanan, L. Nardi, T. Zhao, J. Zhang, P. Bailis, K. Olukotun, C. Ré, and M. Zaharia, "Analysis of the time-to-accuracy metric and entries in the dawnbench deep learning benchmark."